

Multi Networking

Overview

일반적인 Kubernetes Pod에는 `lo` 를 제외하면 하나의 네트워크 인터페이스만 존재합니다. 보통 `eth0` 라는 이름을 가지며, Pod 간 통신이나 Service 접근과 같은 Kubernetes의 기본 네트워킹을 담당합니다.

하지만 모든 워크로드가 하나의 네트워크만 필요로 하는 것은 아닙니다.

- 애플리케이션 트래픽과 관리 트래픽을 분리해야 하는 경우
- 통신사 CNF처럼 제어망과 데이터망을 별도로 사용해야 하는 경우
- 스토리지 전용 네트워크나 백업 전용 네트워크가 필요한 경우
- 서로 다른 VLAN 또는 물리 NIC에 동시에 연결해야 하는 경우

이러한 Pod를 **Multi-homed Pod** 또는 **Multi-NIC Pod**라고 부릅니다. Kubernetes의 기본 네트워크 모델만으로는 Pod에 여러 네트워크 인터페이스를 선언하는 표준 API가 충분하지 않기 때문에, 이 영역에서 주로 사용되는 것이 **Multus CNI**입니다.

이번 글에서는 다음 순서로 Multi-Networking을 살펴보겠습니다.

1. CNI와 Kubernetes 기본 네트워크
2. Multus의 역할과 구조
3. Primary Network와 Secondary Network의 관계
4. CNI Chaining과 Multus Delegation의 차이
5. NetworkAttachmentDefinition
6. macvlan, ipvlan, SR-IOV 비교
7. Multi-NIC Pod 구성
8. Pod 생성 과정과 실제 Packet Flow

Multus?

일반적인 Kubernetes 환경에서는 하나의 CNI 플러그인이 모든 Pod에 기본 네트워크 인터페이스를 만들어 줍니다. Calico, Cilium, Flannel, OVN-Kubernetes 등이 여기에 해당합니다. Multus 문서에서는 이 네트워크를 **Default Network**라고 부르며, 흔히 **Primary Network**, **Primary CNI**라고도 표현합니다.

Multus를 설치하면 CNI 호출 구조의 맨 앞에 Multus가 위치합니다. 이후 Multus는 다음 작업을 조정합니다.

1. 기존 기본 CNI를 호출해 Primary Network 인터페이스를 생성합니다.
2. Pod가 추가 네트워크를 요청했는지 확인합니다.
3. 요청된 NetworkAttachmentDefinition 을 조회합니다.
4. 각 정의에 적힌 CNI 플러그인을 호출해 Secondary Network 인터페이스를 생성합니다.

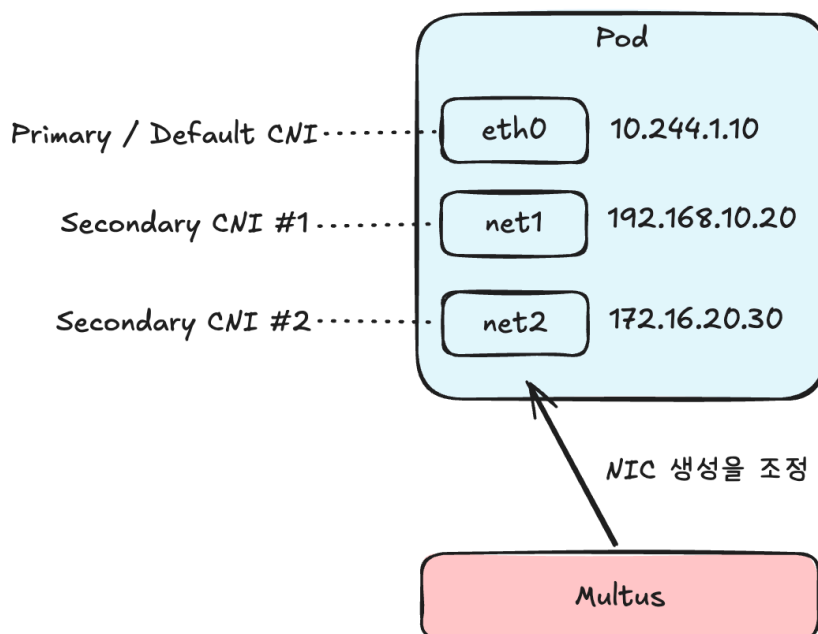
즉 관계를 한 줄로 정리하면 다음과 같습니다.

Multus는 기존 CNI가 만드는 **Primary Network**를 유지하면서, 하나 이상의 **Secondary Network**를 같은 Pod에 추가하는 CNI 메타 플러그인입니다.

공식 Multus 문서도 Default Network를 Pod 간 통신을 담당하고 모든 Pod에 붙는 네트워크로 설명하며, Multus가 만드는 추가 attachment는 이 Default Network에 더해진다고 설명합니다.

Multus Quickstart

구조를 간단히 표현하면 다음과 같습니다.



다만, Multus 자체가 보통 실제 데이터 경로를 구현하는 것은 아니라고 합니다. Multus의 핵심 역할은 다른 CNI 플러그인을 선택하고 호출하는 오케스트레이터에 가깝습니다. 실제 인터페이스 생성과 패킷 전달은 Calico, Cilium, macvlan, ipvlan, SR-IOV CNI 등의 delegate 플러그인이 담당합니다.

또한 `Primary` 와 `Secondary` 는 인터페이스의 물리적 성능이나 중요도를 뜻하지 않습니다.
Kubernetes의 기본 **Pod** 네트워크 역할을 담당하는가, 아니면 그 외의 추가 **attachment**인가를 구분하는 용어에 가깝습니다.

CNI(Container Network Interface)

CNI에 대해서는 2주차에서 다루었지만 간단하게 다시 복습해보겠습니다.

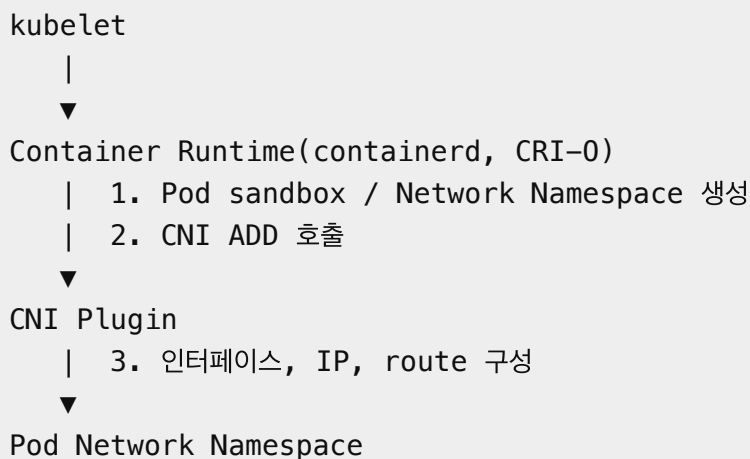
CNI는 컨테이너 런타임과 네트워크 플러그인 사이의 호출 규약입니다. 컨테이너 런타임은 먼저 Pod sandbox의 Network Namespace를 만든 뒤, CNI 플러그인에 `ADD` 요청을 보내 인터페이스와 IP, 라우트 등을 구성하도록 요청합니다.

CNI 명세가 정의하는 핵심은 다음과 같습니다.

- 네트워크 설정을 표현하는 JSON 형식
- 런타임이 플러그인을 실행하는 방법
- `ADD` , `DEL` , `CHECK` , `GC` , `VERSION` 등의 명령
- 여러 플러그인 사이에 결과를 전달하는 방법
- 플러그인이 인터페이스, IP, route, DNS 정보를 반환하는 형식

CNI 명세에서 `ADD` 호출은 `CNI_NETNS` , `CNI_IFNAME` , `CNI_CONTAINERID` 등의 정보를 플러그인에 전달합니다. 플러그인은 해당 Network Namespace 안에 요청된 인터페이스를 만들거나 설정합니다. [CNI Specification](#)

일반적인 Pod 생성 과정을 단순화하면 다음과 같습니다.



Kubernetes에서 CNI 플러그인은 Kubernetes 네트워크 모델을 구현하는 역할을 합니다.

일반적인 단일 네트워크 구성

Multus가 없는 일반적인 클러스터에서는 컨테이너 런타임이 노드의 CNI 설정을 읽고 기본 CNI를 호출합니다.

예를 들어 기본 CNI가 Calico라면 다음과 같은 구조가 됩니다.

Runtime → CNI ADD → Calico → Pod eth0 생성 및 IP/route 설정

이 구조에서 Pod의 `eth0`는 다음 역할을 담당합니다.

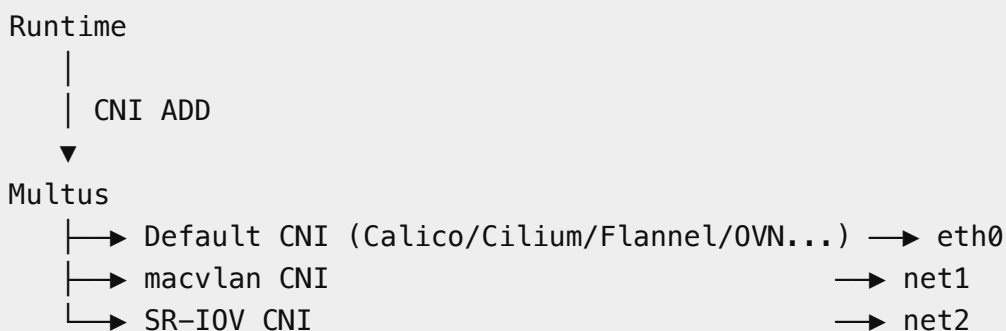
- 다른 Pod와의 통신
- Kubernetes Service 접근
- DNS 서버(CoreDNS) 접근
- Kubernetes API Server 등 클러스터 서비스 접근
- 일반적인 외부 네트워크 통신

이 네트워크가 **Primary Network(Default Network)**입니다.

Multus CNI Architecture

Multus는 여러 CNI 플러그인을 호출할 수 있는 **CNI meta-plugin**입니다. 이름의 의미도 여러 네트워크를 하나의 Pod에 붙인다는 특성에서 이해할 수 있습니다.

Multus가 설치된 환경에서는 호출 흐름이 다음과 같이 바뀝니다.



Multus 공식 프로젝트도 Multus를 여러 CNI를 호출하여 Pod에 여러 인터페이스를 붙이는 meta-plugin으로 정의합니다. [Multus CNI](#)

Multus가 하는 일과 하지 않는 일

구분	내용
Multus가 하는 일	기본 CNI 호출, Pod annotation 해석, NAD 조회, 추가 CNI 호출, 결과 취합
Delegate CNI가 하는 일	인터페이스 생성, IP 설정, route 설정, VLAN/VF 설정 등
Linux 커널이 하는 일	Pod route lookup, ARP/NDP, 실제 패킷 송수신
Multus가 일반적으로 하지 않는 일	실행 중인 모든 패킷을 프록시하거나 포워딩하는 작업

Pod 생성이 완료된 뒤 패킷이 `eth0` 또는 `net1` 중 어느 인터페이스로 나갈지는 Multus가 실시간으로 선택하지 않습니다. Pod Network Namespace 내부의 라우팅 테이블과 **policy routing rule**이 결정합니다.

Thin Plugin과 Thick Plugin

Multus 4.0부터는 기존 구조를 **thin plugin**, 새 client/server 구조를 **thick plugin**이라고 구분합니다.

- **Thin plugin:** CNI 호출 때 Multus 바이너리가 직접 주요 로직을 수행합니다.
- **Thick plugin:** 노드마다 `multus-daemon` 이 실행되고, 가벼운 shim이 daemon과 통신합니다. metrics 등 thin 구조에 없던 기능도 지원합니다.

두 방식 모두 이 글에서 설명하는 Primary/Secondary Network 모델은 동일합니다. 차이는 Multus의 배포 구조와 부가 기능에 있습니다. [Multus CNI](#)

Thin Plugin v.s Thick Plugin

With the multus 4.0 release, we introduce a new client/server-style plugin deployment. This new deployment is called '**thick plugin**', in contrast to deployments in previous versions, which is now called a 'thin plugin'. The new thick plugin consists of two binaries, multus-daemon and multus-shim CNI plugin.

The 'multus-daemon' will be deployed to all nodes as a local agent and supports additional features, such as metrics, which were not available with the 'thin plugin' deployment before.

Due to these additional features, the 'thick plugin' comes with the trade-off of consuming more resources than the 'thin plugin'.

Thick plugin에서 추가되는 운영 기능은 다음과 같습니다.

- **Metrics endpoint:** `metricsPort` 를 설정하면 Multus metric exporter를 노출할 수 있습니다. 이를 통해 Multus가 제공하는 지표를 모니터링 시스템에서 수집할 수 있습니다.
- **지속적인 daemon 로그:** 일회성으로 실행되는 CNI 프로세스와 달리 노드별 daemon이 계속 실행되므로, `logFile`, `logLevel`, `logToStderr` 를 통해 Multus의 동작과 delegate 호출 오류를 한곳에서 추적하기 쉽습니다.
- **daemon 중심 설정:** socket 위치, host root 경로, CNI binary와 configuration directory 등을 daemon 설정에서 관리할 수 있습니다. daemon 설정은 shim 쪽 Multus 설정을 덮어쓸 수 있습니다.
- **Auxiliary CNI chain 실행:** `auxiliaryCNIChainName` 을 사용하면 노드 디스크에 저장된 별도의 CNI configuration chain을 추가로 실행할 수 있습니다.

Primary Network vs Secondary Network

Primary Network(Default Network)

Primary Network는 Kubernetes의 기본 네트워크 모델을 구현하는 네트워크입니다. 일반적으로 모든 Pod에 자동으로 연결되며 인터페이스 이름은 `eth0` 입니다.

대표적으로 다음 CNI들이 Primary Network 역할을 합니다.

- Calico
- Cilium
- Flannel

Primary Network에는 보통 다음 특성이 있습니다.

항목	설명
연결 대상	일반적으로 모든 Pod
인터페이스 이름	주로 <code>eth0</code>
주요 목적	Pod-to-Pod, Service, DNS, 클러스터 제어 통신
생성 시점	Pod sandbox 네트워크 구성 시 자동 생성
기본 경로	일반적으로 Primary Network 쪽에 존재
Kubernetes NetworkPolicy	기본 CNI가 구현하는 범위에서 적용

Multus 설정 관점에서 이 네트워크는 `clusterNetwork` 또는 기본 delegate로 지정될 수 있습니다. Multus 설정 문서도 Default Network를 Kubernetes networking model을 구현하는 CNI로 정의합니다. [Multus Configuration Reference](#)

Secondary Network(Additional Network)

Secondary Network는 Primary Network 이외에 Pod에 추가로 연결되는 네트워크입니다. 보통 Pod annotation에서 원하는 네트워크를 명시적으로 선택합니다.

대표적인 사용 목적은 다음과 같습니다.

- 데이터 플레인 전용 네트워크
- 스토리지 전용 네트워크
- 관리망과 서비스망 분리
- 외부 VLAN 직접 연결
- 서로 다른 물리 NIC를 사용하는 이중화 구성

인터페이스 이름은 기본적으로 `net1`, `net2` 처럼 생성되며 annotation에서 이름을 지정할 수도 있습니다.

```
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: |-
      [
        { "name": "data-net", "interface": "data0" },
        { "name": "storage-net", "interface": "storage0" }
      ]
```

Primary와 Secondary의 차이

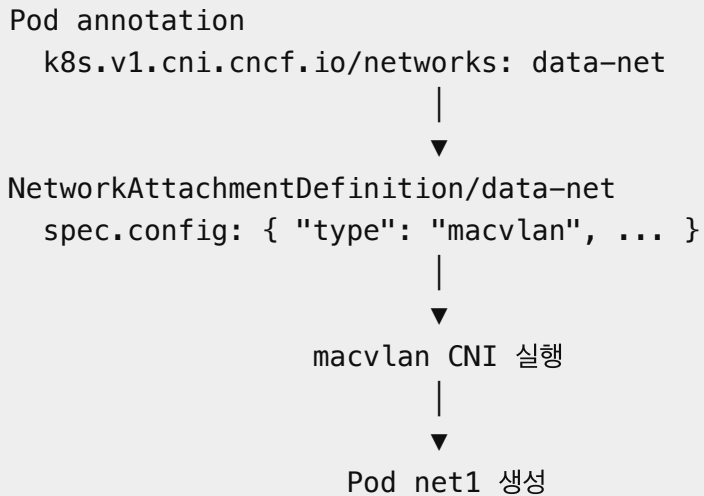
항목	Primary Network	Secondary Network
다른 이름	Default Network, Cluster Network	Additional Network
기본 연결 여부	일반적으로 모든 Pod에 자동 연결	요청한 Pod에만 연결
대표 인터페이스	<code>eth0</code>	<code>net1</code> , <code>net2</code> 또는 사용자 지정 이름
정의 위치	노드의 기본 CNI/Multus 설정	주로 NetworkAttachmentDefinition
기본 용도	Kubernetes 기본 연결성	목적별 별도망, VLAN, 고성능 NIC
기본 route	일반적으로 보유	기본적으로 필수 아님
장애 영향	DNS, Service 등 기본 통신 전체에 영향 가능	해당 추가망을 사용하는 기능에 주로 영향

NetworkAttachmentDefinition

`NetworkAttachmentDefinition`, 줄여서 NAD는 Secondary Network의 연결 방법을 Kubernetes API 리소스로 표현하기 위한 CRD입니다.

Multus는 Pod annotation에 적힌 네트워크 이름을 보고 같은 namespace 또는 명시된 다른 namespace에서 NAD를 찾습니다. 그다음 NAD의 `spec.config`에 저장된 CNI 설정을 읽어 해당 플러그인을 호출합니다.

구조는 다음과 같습니다.



NAD API의 핵심 필드는 문자열 형태의 `spec.config`이며, 이 안에 CNI JSON 설정이 저장됩니다. [Network Attachment Definition API](#)

macvlan NAD 예시

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: data-net
  namespace: default
spec:
  config: |-
    {
      "cniVersion": "1.0.0",
      "name": "data-net",
      "type": "macvlan",
      "master": "ens192",
      "mode": "bridge",
      "ipam": {
        "type": "host-local",
        "subnet": "192.168.100.0/24",
        "rangeStart": "192.168.100.100",
        "rangeEnd": "192.168.100.200",
```

```

    "gateway": "192.168.100.1"
  }
}

```

각 필드의 의미는 다음과 같습니다.

필드	의미
type	실행할 CNI 플러그인
master	macvlan이 연결될 노드의 상위 인터페이스
mode	macvlan 동작 모드
ipam.type	IP를 할당할 IPAM 플러그인
subnet	Secondary Network의 IP 대역
rangeStart , rangeEnd	할당 범위
gateway	해당 네트워크의 게이트웨이

Pod에서 NAD 선택하기

```

apiVersion: v1
kind: Pod
metadata:
  name: multi-nic-pod
  annotations:
    k8s.v1.cni.cncf.io/networks: data-net # NAD 이름
spec:
  containers:
    - name: app
      image: nicolaka/netshoot
      command: ["sleep", "infinity"]

```

Pod가 생성되면 다음과 같은 결과를 기대할 수 있습니다.

```

$ kubectl exec multi-nic-pod -- ip -br addr
lo                UNKNOWN        127.0.0.1/8
eth0@if12         UP             10.244.1.15/24
net1@ens192       UP             192.168.100.100/24

```

여러 NAD 선택하기

간단한 문자열 형식도 사용할 수 있습니다.

```
annotations:
  k8s.v1.cni.cncf.io/networks: data-net,storage-net
```

namespace나 인터페이스 이름 등 세부 설정이 필요하다면 JSON 형식이 더 명확합니다.

```
annotations:
  k8s.v1.cni.cncf.io/networks: |-
    [
      {
        "name": "data-net",
        "namespace": "network-system",
        "interface": "data0"
      },
      {
        "name": "storage-net",
        "namespace": "network-system",
        "interface": "storage0"
      }
    ]
```

Network Status 확인

Multus는 연결 결과를 일반적으로 다음 annotation에 기록합니다.

```
kubectl get pod multi-nic-pod \
  -o jsonpath='{.metadata.annotations.k8s.v1.cni.cncf.io/network-status}'
```

예시는 다음과 같습니다.

```
[
  {
    "name": "default/cluster-network",
    "interface": "eth0",
    "ips": ["10.244.1.15"],
    "default": true
  },
  {
    "name": "default/data-net",
    "interface": "net1",
    "ips": ["192.168.100.100"]
  }
]
```

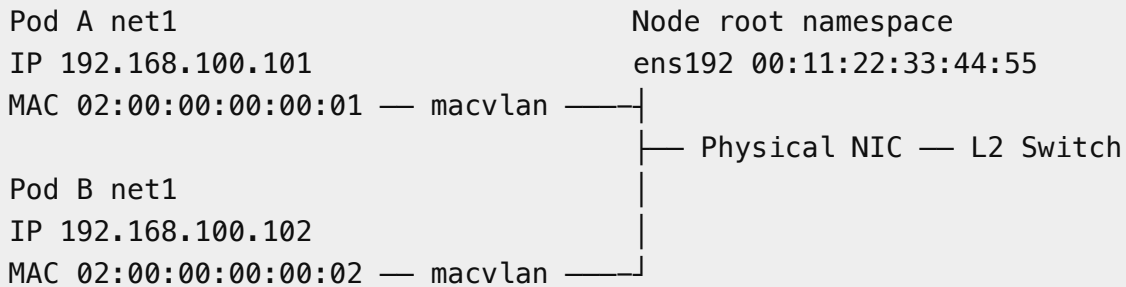
```
}  
]
```

Secondary Network용 CNI 비교

Multus는 Secondary Network를 직접 구현하는 것이 아니라, 목적에 맞는 CNI 플러그인을 호출합니다. 그중 대표적인 방식이 macvlan, ipvlan, SR-IOV입니다.

macvlan

macvlan은 노드의 물리 인터페이스 또는 VLAN 인터페이스를 **master**로 사용하고, 그 아래에 여러 개의 가상 인터페이스를 생성합니다. 각 macvlan 인터페이스는 서로 다른 MAC 주소를 가지며 Pod의 Network Namespace로 이동합니다.



Multus가 macvlan CNI를 호출하면 플러그인은 대략 다음 작업을 수행합니다.

1. NAD의 **master**로 지정된 노드 인터페이스를 찾습니다.
2. 해당 인터페이스를 parent로 하는 macvlan link를 생성합니다.
3. 생성한 link를 Pod Network Namespace로 이동하고 **net1** 또는 사용자가 요청한 이름으로 변경합니다.
4. IPAM 플러그인을 호출해 IP 주소와 route를 할당받습니다.
5. Pod 내부 macvlan 인터페이스에 IP, route, MAC, MTU를 설정하고 link를 활성화합니다.

192.168.100.101을 가진 Pod가 같은 subnet의 192.168.100.50으로 패킷을 보낼 때는 먼저 **net1**에서 ARP Request를 전송합니다. ARP frame의 source MAC은 Pod macvlan 인터페이스의 MAC입니다. macvlan driver는 이 frame을 **ens192**를 통해 물리 NIC로 전달합니다. 외부 스위치는 하나의 노드 포트에서 여러 Pod MAC을 학습하고, ARP Reply와 이후 unicast frame을 각 MAC 주소를 기준으로 다시 전달합니다.

노드가 수신한 frame의 destination MAC이 특정 macvlan 인터페이스와 일치하면 macvlan driver가 해당 frame을 그 인터페이스가 있는 Pod Network Namespace로 전달합니다. 따라서 같은 물리 NIC를 사용하더라도 각 Pod는 자신에게 할당된 MAC과 IP를 가진 별도 호스트처럼 기존

L2 network에 참여합니다. 별도의 veth pair나 Linux bridge를 경유하지 않는다는 점도 일반적인 bridge CNI 경로와 다른 부분입니다.

macvlan 사용이 불가능한 경우

macvlan은 노드 간 네트워크를 새로 만들어 주는 overlay 기술이 아니라, 이미 구성된 물리 네트워크를 그대로 사용합니다. 따라서 같은 subnet의 macvlan Pod가 서로 다른 노드에 있다면 각 노드의 master 인터페이스가 동일한 L2 브로드캐스트 도메인, 일반적으로 같은 VLAN에 연결되어 있어야 합니다. 반드시 같은 물리 스위치일 필요는 없지만 스위치 사이에서 해당 VLAN이 전달되어야 합니다.

Pod가 ARP Request를 보내면 스위치는 이를 같은 VLAN 전체로 전달합니다. 목적지 Pod가 응답하면 스위치는 각 Pod의 MAC 주소가 어느 노드의 물리 포트 뒤에 있는지 FDB에 기록하고, 이후 frame을 해당 포트로 전달합니다.

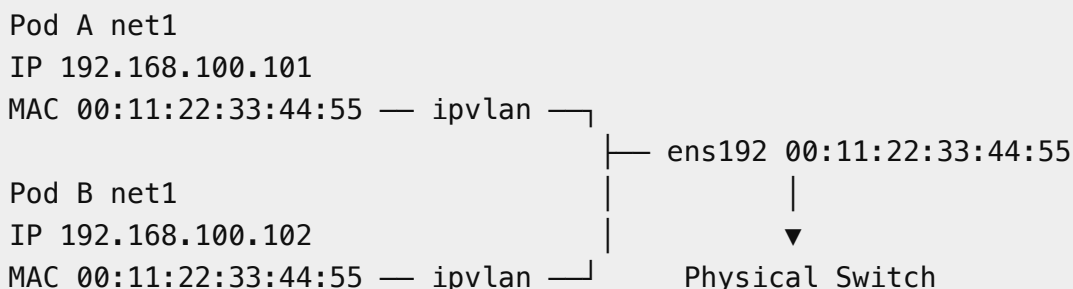
다음 환경에서는 macvlan을 그대로 사용할 수 없습니다.

- 같은 subnet의 Pod가 서로 다른 VLAN에 있어 ARP broadcast가 전달되지 않는 경우
- 스위치의 Port Security가 한 포트에서 여러 MAC 주소를 허용하지 않는 경우
- Hypervisor나 클라우드 네트워크가 VM에 등록되지 않은 Pod MAC의 송수신을 차단하는 경우
- 여러 MAC 주소 사용을 지원하지 않는 무선 NIC를 master 로 사용하는 경우
- NAD에 지정한 master 인터페이스가 Pod가 배치된 노드에 존재하지 않는 경우

또한 macvlan의 parent 인터페이스와 그 아래의 macvlan 인터페이스는 직접 통신할 수 없습니다. 따라서 노드와 macvlan Pod가 통신해야 한다면 Kubernetes Primary Network를 사용하거나, 노드에도 별도의 macvlan 인터페이스를 구성해야 합니다.

ipvlan

ipvlan도 노드 인터페이스를 master 로 사용해 Pod용 가상 인터페이스를 생성하지만, macvlan과 달리 모든 ipvlan 인터페이스가 master의 MAC 주소를 공유합니다. Pod마다 IP 주소는 다르지만 외부 네트워크에 보이는 source MAC은 같습니다.



Multus가 ipvlan CNI를 호출하면 플러그인은 master 아래에 ipvlan link를 생성하고 이를 Pod Network Namespace로 이동한 뒤 IPAM 결과를 적용합니다. 인터페이스 생성 과정은 macvlan과 비슷하지만 수신 패킷을 Pod에 분배하는 기준이 다릅니다. macvlan은 destination MAC을 기준으로 child interface를 찾지만, ipvlan은 destination IP를 기준으로 해당 slave를 찾습니다.

12 모드에서 192.168.100.101 인 Pod가 같은 subnet의 장비로 패킷을 보내면 Pod는 ARP를 수행하고 Ethernet frame을 생성합니다. 이때 frame에는 master와 공유하는 MAC이 사용됩니다. ipvlan driver가 frame을 master로 넘기고 ens192 를 통해 외부로 전송합니다. 외부 스위치는 노드 포트에서 하나의 MAC만 주로 학습하지만, upstream 장비의 ARP table에는 서로 다른 Pod IP가 같은 MAC에 대응할 수 있습니다. 수신 frame이 노드에 도착하면 ipvlan driver가 destination IP를 확인하여 해당 Pod의 net1 로 전달합니다.

근데 같은 MAC 주소인데 어떻게 프레임이 목적지 Pod에 도착할 수 있죠?

MAC 주소는 외부 스위치가 프레임을 노드까지 전달할 때 사용하고, 목적지 IP는 노드의 ipvlan 드라이버가 프레임을 Pod의 ipvlan 인터페이스로 전달할 때 사용합니다.

예를 들어 외부 장비가 Pod A의 192.168.100.101 로 패킷을 보내면 프레임에는 다음 정보가 들어 있습니다.

```
Destination MAC: 00:11:22:33:44:55
Destination IP: 192.168.100.101
```

처리 과정은 다음과 같습니다.

1. 외부 스위치는 destination MAC을 보고 프레임을 ens192 가 연결된 노드로 전달합니다.
2. 노드의 ens192 가 프레임을 수신합니다.
3. ipvlan 드라이버가 프레임 내부의 destination IP를 확인합니다.
4. 192.168.100.101 이 설정된 ipvlan 인터페이스를 찾습니다.
5. 해당 인터페이스가 있는 Pod A의 Network Namespace로 프레임을 전달합니다.

```
External Host
|
| dst MAC = 00:11:22:33:44:55
| dst IP  = 192.168.100.101
▼
Physical Switch
| MAC을 기준으로 노드 선택
▼
Node ens192
|
| ipvlan driver가 목적지 IP 확인
```

▼
192.168.100.101 → Pod A의 net1

즉 `ipvlan`에서는 모든 Pod가 같은 MAC 주소를 사용해도 문제가 되지 않습니다.

외부 스위치 → 노드 : MAC 주소로 전달
노드 → `ipvlan` 인터페이스 : IP 주소로 전달

`macvlan`이 Pod별 MAC 주소로 인터페이스를 구분한다면, `ipvlan`은 공유 MAC으로 프레임을 노드까지 받은 다음 목적지 IP를 기준으로 각 Pod의 `ipvlan` 인터페이스를 구분합니다.

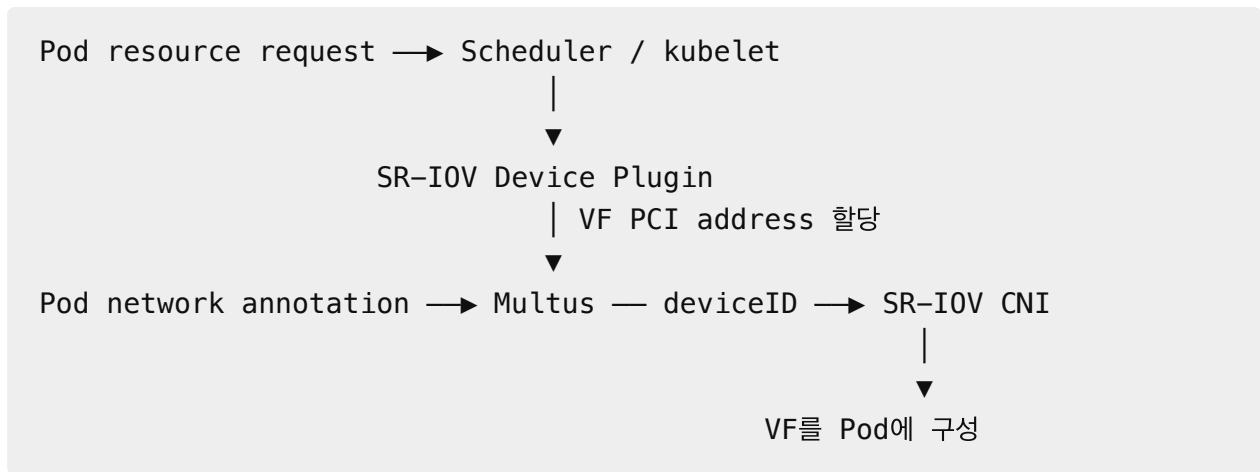
SR-IOV

SR-IOV는 하나의 물리 PCIe NIC를 **Physical Function(PF)** 과 여러 개의 **Virtual Function(VF)** 으로 나누는 하드웨어 가상화 기술입니다. PF는 노드에서 NIC 전체와 VF 설정을 관리하는 기능을 가지며, 각 VF는 독립적인 PCI function과 송수신 queue를 가집니다.

```
Physical NIC / Port
├─ PF enp6s0f0 (host에서 VF, VLAN, MAC 등을 관리)
│   ├── VF 0 / PCI 0000:41:02.0 → Pod A sriov0
│   ├── VF 1 / PCI 0000:41:02.1 → Pod B sriov0
│   └─ VF 2 / PCI 0000:41:02.2 → 사용 가능 자원
```

SR-IOV CNI만으로는 여러 Pod 중 어느 Pod에 어느 VF를 줄지 결정할 수 없습니다. Kubernetes에서는 보통 SR-IOV Network Device Plugin과 함께 사용합니다.

1. 노드에서 PF를 통해 필요한 수만큼 VF를 생성합니다.
2. SR-IOV Device Plugin이 사용 가능한 VF를 검색하여 `intel.com/sriov_netdevice` 같은 extended resource로 kubelet에 등록합니다.
3. Pod가 이 resource를 request하면 Scheduler는 해당 VF 자원이 남아 있는 노드를 선택합니다.
4. kubelet과 Device Plugin이 특정 VF를 Pod에 할당합니다.
5. Multus는 NAD의 `k8s.v1.cni.cncf.io/resourceName` 과 Pod에 할당된 자원을 대응시켜 VF의 PCI 주소인 `deviceID` 를 찾습니다.
6. Multus가 `deviceID` 와 함께 SR-IOV CNI를 호출합니다.
7. SR-IOV CNI는 PF를 통해 VF의 MAC, VLAN, spoof check, trust 등의 값을 설정하고, kernel driver를 사용하는 VF라면 VF의 netdevice를 Pod Network Namespace로 이동해 `sriov0` 로 이름을 변경합니다.
8. IPAM 설정이 있으면 IP 주소와 route도 `sriov0` 에 적용합니다.



Pod가 `sriov0` 로 패킷을 보내면 패킷은 VF driver와 VF 전용 queue를 통해 NIC로 전달됩니다. NIC는 VF에 설정된 MAC과 VLAN 정책을 적용하고 물리 포트로 frame을 전송합니다. 반대 방향의 frame도 NIC가 MAC, VLAN 등의 조건으로 VF를 선택하여 해당 VF queue에 전달합니다. veth pair, Linux bridge 또는 host의 software switch를 통과하는 일반적인 경로가 필요하지 않습니다.

여기서 kernel driver를 사용하는 SR-IOV 인터페이스는 여전히 Pod의 Linux network stack을 거칩니다. 반면 VF가 `vfio-pci` 와 같은 userspace driver에 bind되어 DPDK 애플리케이션에서 사용되면, 일반 Linux 인터페이스와 kernel TCP/IP stack을 통하지 않고 애플리케이션이 VF queue를 직접 처리합니다. 이 경우 CNI의 IPAM은 주소를 예약할 수는 있어도 Linux 인터페이스에 IP를 설정하지는 않습니다.

SR-IOV는 통신사 UPF와 같은 CNF 데이터 플레인, 낮은 latency가 필요한 네트워크 기능, 고속 패킷 처리, HPC와 AI/ML의 고대역폭 통신에 주로 사용됩니다. VF는 실제 하드웨어 자원이므로 NIC 지원 여부, VF 개수, NUMA 위치, driver와 IOMMU 설정을 함께 관리해야 합니다. 또한 트래픽이 Primary CNI의 datapath를 지나지 않을 수 있으므로 NetworkPolicy와 관찰 기능이 SR-IOV 경로에도 적용되는지 별도로 확인해야 합니다. [SR-IOV CNI](#), [SR-IOV Network Device Plugin](#)

비교 정리

항목	macvlan	ipvlan	SR-IOV
생성되는 인터페이스	master 아래의 macvlan link	master 아래의 ipvlan link	NIC가 제공하는 VF의 netdevice 또는 PCI device
Pod별 MAC	서로 다른 MAC 사용	master MAC 공유	VF별 MAC 설정 가능
수신 분배 기준	destination MAC	destination IP	NIC hardware의 VF 분류
외부 네트워크 연결 방식	기존 L2 segment에 직접 참여	L2 참여 또는 L3 routing	PF가 연결된 물리 포트를 통해 송수신
별도 자원 스케줄링	일반적으로 불필요	일반적으로 불필요	Device Plugin 또는 DRA 필요
대표적인 용도	기존 VLAN/L2망, DHCP 기반 네트워크	MAC 제한 환경, routed Pod network	CNF, DPDK, 저지연·고처리량 통신

Multi-NIC Pod 구성 예시

이번에는 Primary Network 외에 macvlan 기반 데이터망과 SR-IOV 기반 고성능망을 함께 사용하는 Pod를 가정해보겠습니다.

macvlan NetworkAttachmentDefinition

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: data-net
spec:
  config: |-
    {
      "cniVersion": "1.0.0",
      "name": "data-net",
      "type": "macvlan",
      "master": "ens192",
      "mode": "bridge",
      "ipam": {
        "type": "host-local",
        "subnet": "192.168.100.0/24",
        "rangeStart": "192.168.100.100",
        "rangeEnd": "192.168.100.200",
        "gateway": "192.168.100.1"
      }
    }
  }
```

SR-IOV NetworkAttachmentDefinition

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: sriov-net
  annotations:
    k8s.v1.cni.cncf.io/resourceName: intel.com/sriov_netdevice
spec:
  config: |-
    {
      "cniVersion": "1.0.0",
      "name": "sriov-net",
      "type": "sriov",
      "vlan": 200,
      "ipam": {
```

```

    "type": "host-local",
    "subnet": "172.16.200.0/24",
    "rangeStart": "172.16.200.100",
    "rangeEnd": "172.16.200.200",
    "gateway": "172.16.200.1"
  }
}

```

`resourceName` 은 실제 SR-IOV Device Plugin 설정과 일치해야 합니다. SR-IOV CNI 공식 예제도 NAD annotation을 이용해 Device Plugin resource와 네트워크를 연결합니다. [SR-IOV CNI](#)

Multi-NIC Pod

```

apiVersion: v1
kind: Pod
metadata:
  name: multi-nic-pod
  annotations:
    k8s.v1.cni.cncf.io/networks: |-
      [
        { "name": "data-net", "interface": "data0" },
        { "name": "sriov-net", "interface": "sriov0" }
      ]
spec:
  containers:
    - name: app
      image: nicolaka/netshoot
      command: ["sleep", "infinity"]
      resources:
        requests:
          intel.com/sriov_netdevice: "1"
        limits:
          intel.com/sriov_netdevice: "1"

```

Pod 내부에서는 다음과 같은 인터페이스를 기대할 수 있습니다.

eth0	10.244.1.20/24	Primary Network
data0	192.168.100.101/24	macvlan Secondary Network
sriov0	172.16.200.101/24	SR-IOV Secondary Network

확인은 다음 명령으로 할 수 있습니다.

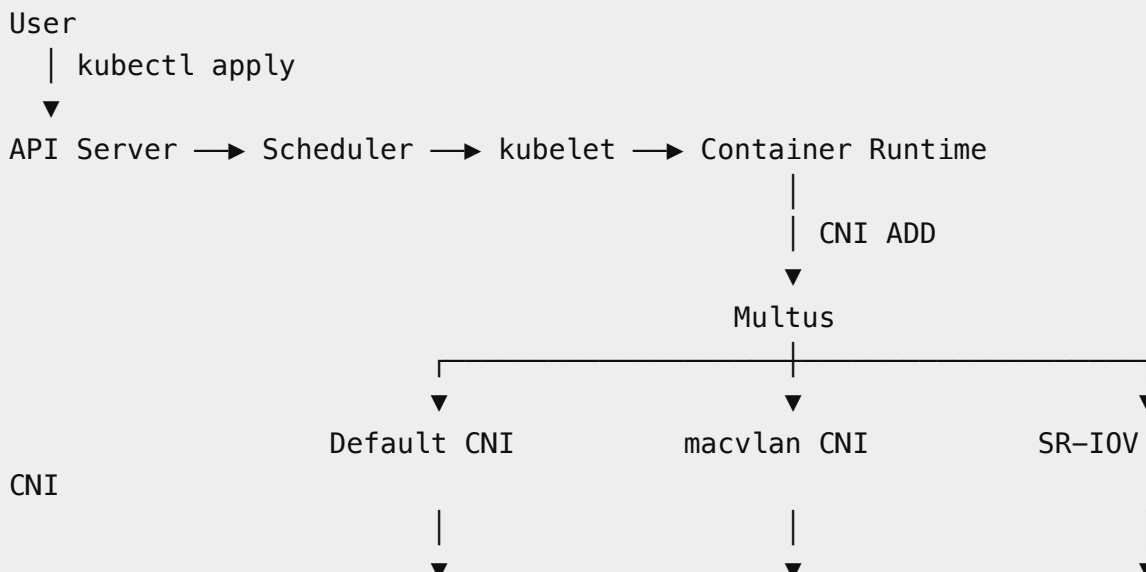
```
kubectl exec multi-nic-pod -- ip -br link
kubectl exec multi-nic-pod -- ip -br addr
kubectl exec multi-nic-pod -- ip route
kubectl exec multi-nic-pod -- ip rule
```

Pod 생성 시 Control Flow

앞의 YAML을 생성했을 때 내부 동작을 순서대로 살펴보겠습니다.

일반적인 생성 순서

1. 사용자가 Pod를 생성합니다.
2. Scheduler가 Pod를 실행할 노드를 결정합니다.
3. kubelet이 container runtime에 Pod sandbox 생성을 요청합니다.
4. runtime이 Pod의 Network Namespace를 생성합니다.
5. runtime이 노드의 CNI 설정에 따라 Multus를 호출합니다.
6. Multus가 기본 CNI를 delegate로 호출합니다.
7. 기본 CNI가 `eth0` 와 Primary Network IP, route를 구성합니다.
8. Multus가 Pod의 `k8s.v1.cni.cncf.io/networks` annotation을 읽습니다.
9. Multus가 `data-net` 과 `sriov-net` NAD를 조회합니다.
10. Multus가 macvlan CNI를 호출하여 `data0` 를 생성합니다.
11. Multus가 할당된 VF의 `deviceID` 와 함께 SR-IOV CNI를 호출하여 `sriov0` 를 구성합니다.
12. 각 IPAM 플러그인이 IP 주소를 할당합니다.
13. Multus가 결과를 취합하고 `network-status` annotation을 갱신합니다.
14. 네트워크 구성이 성공하면 Pod의 애플리케이션 컨테이너가 시작됩니다.



	eth0	data0
sriov0		

Secondary Network 중 하나의 생성에 실패하면 Pod sandbox 네트워크 구성이 실패할 수 있습니다. 이 경우 Pod가 `ContainerCreating` 에 머물고, event에 CNI 또는 Multus 오류가 나타나는 경우가 많습니다.

Default Route는 어느 인터페이스를 사용하는가?

Secondary Network가 추가되어도 일반적으로 기존 default route는 `eth0` 에 남습니다.

```
default via 10.244.1.1 dev eth0
```

따라서 Secondary Network와 직접 연결된 subnet 또는 명시적으로 추가한 route가 아니라면 패킷은 계속 Primary Network로 나갑니다.

특정 Secondary Network를 default route로 만들고 싶다면 Multus의 JSON annotation에서 `default-route` 를 지정할 수 있습니다.

```
annotations:
  k8s.v1.cni.cncf.io/networks: |-
    [
      {
        "name": "data-net",
        "interface": "data0",
        "default-route": ["192.168.100.1"]
      }
    ]
```

Multus 사용 가이드도 일반적으로 default route는 `eth0` 를 사용하지만, 필요하면 특정 attachment에 `default-route` 를 지정할 수 있다고 설명합니다. 동시에 이 설정이 cluster-wide default network를 통한 연결성에 영향을 줄 수 있다고 경고합니다. [Multus Usage Guide](#)

이 설정은 주의해서 사용해야 합니다. default route를 Secondary Network로 바꾸면 다음 문제가 생길 수 있습니다.

- Kubernetes API Server 접근 실패
- CoreDNS 응답의 return path 불일치
- Service 또는 다른 Pod로 가는 경로 변경

- asymmetric routing
- reverse path filtering에 의한 packet drop
- 동일 목적지에 대한 route 우선순위 충돌

프로덕션 환경에서는 default route를 단순 교체하기보다 목적지 대역별 static route나 source-based policy routing을 사용하는 경우가 많습니다.

예를 들어 출발지 IP에 따라 경로를 분리할 수 있습니다.

```
ip rule add from 192.168.100.101/32 table 100
ip route add default via 192.168.100.1 dev data0 table 100
```

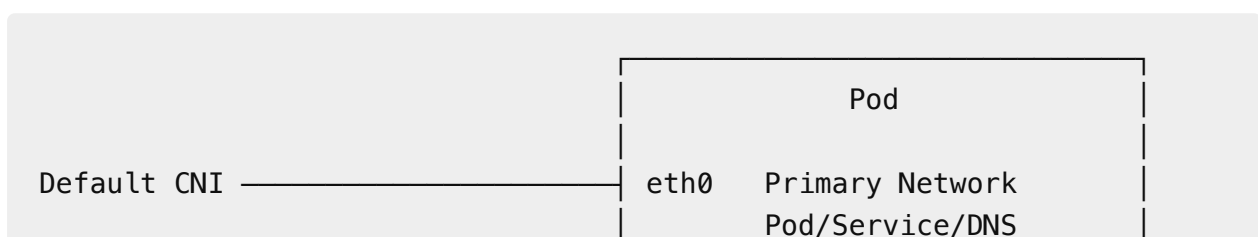
다만 CNI가 생성한 설정을 컨테이너 내부에서 임시로 수정하면 Pod 재생성 시 사라집니다. 실제 환경에서는 CNI 기능, tuning plugin, init container와 필요한 capability, 또는 별도 네트워크 관리 방식을 통해 선언적으로 구성하는 편이 좋습니다.

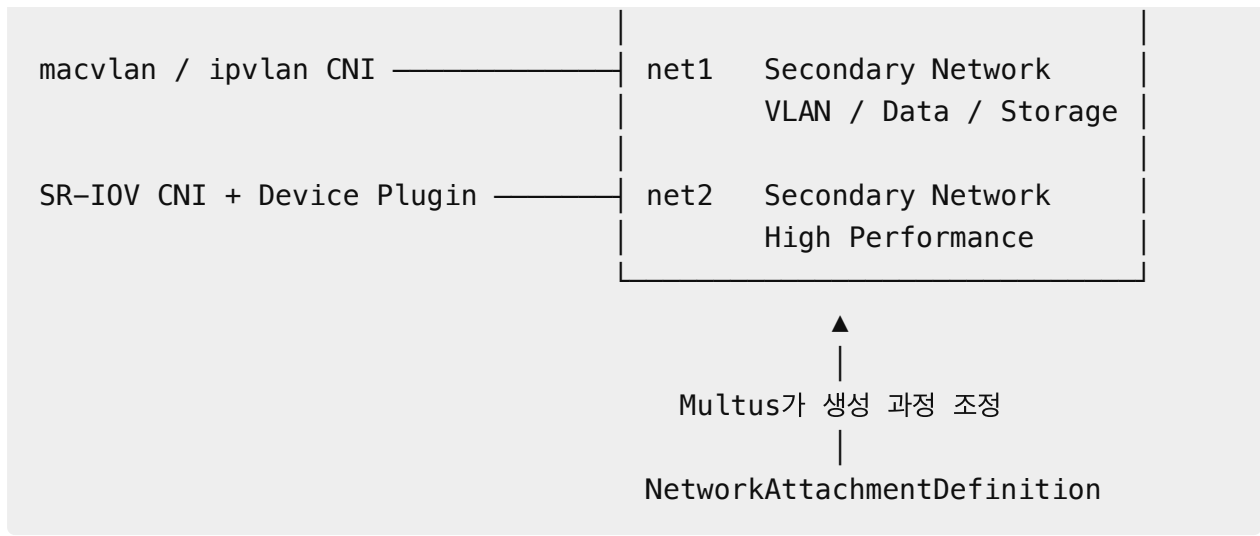
정리

지금까지의 관계를 다시 정리하면 다음과 같습니다.

1. Kubernetes는 CNI를 통해 Pod의 Network Namespace에 네트워크를 구성합니다.
2. 기존 Calico, Cilium, Flannel 등의 CNI가 Kubernetes 기본 연결성을 제공하며, 이를 Primary 또는 Default Network라고 부릅니다.
3. Multus는 CNI meta-plugin으로 동작하며 기본 CNI를 먼저 호출합니다.
4. Pod annotation에 추가 네트워크 요청이 있으면 Multus가 NAD를 조회합니다.
5. NAD의 설정에 따라 macvlan, ipvlan, SR-IOV 등의 CNI를 호출합니다.
6. 그 결과 하나의 Pod 안에 `eth0`, `net1`, `net2` 와 같은 여러 인터페이스가 생성됩니다.
7. Pod 실행 중 실제 packet path는 Multus가 아니라 Linux route와 policy routing이 결정합니다.
8. Secondary Network는 Primary CNI의 NetworkPolicy, observability, NAT 경로를 우회할 수 있으므로 별도의 보안과 운영 설계가 필요합니다.

전체 구조를 한 번 더 표현하면 다음과 같습니다.





기본 CNI 하나로 구성된 **Kubernetes** 네트워크에서 출발해, **Multus**가 그 기본 네트워크를 **Primary**로 유지하고 목적별 **Secondary Network**를 추가한다.

참고 자료

Multus / Network Plumbing

- [Multus CNI — GitHub](#)
- [Multus Quickstart Guide](#)
- [Multus Configuration Reference](#)
- [Multus Usage Guide](#)
- [Network Attachment Definition API](#)

CNI / Kubernetes

- [CNI Specification](#)
- [Kubernetes — Network Plugins](#)
- [CNI Plugins](#)

Secondary Network Plugins

- [CNI macvlan plugin](#)
- [CNI ipvlan plugin](#)
- [Linux Kernel — IPVLAN Driver HOWTO](#)
- [SR-IOV CNI](#)
- [SR-IOV Network Device Plugin](#)